inkor
inkorformacion.com

IFCT032PO
DATA MINING
BUSINESS INTELLIGENCE

23 mayo al 5 junio 2024

SAN SEBASTIÁN · MADRID · BARCELONA · VALENCIA · MÁLAGA · PAMPLONA · LOGROÑO · QUERÉTARO (MX)

Los **árboles de decisión** son modelos predictivos que se utilizan en machine learning y minería de datos para tomar decisiones basadas en varias condiciones.

Imagina un árbol invertido, donde cada nodo interno representa una pregunta sobre una característica (atributo), cada rama representa una posible respuesta a esa pregunta, y cada hoja representa la predicción o resultado. Ejemplo:

Supongamos que queremos predecir si alguien jugará al tenis en función de dos características: el clima y la temperatura.

1. **Nodo Raíz (Pregunta inicial):** ¿Cómo está el clima?
    - Si es "Soleado", seguimos por la rama de la derecha.
    - Si no, seguimos por la rama de la izquierda.
2. **Nodo Secundario (Pregunta sobre Temperatura):** ¿La temperatura es mayor a 25 grados?
    - Si es "Sí", entonces llegamos a una hoja que dice "Sí, jugará al tenis".
    - Si es "No", llegamos a otra hoja que dice "No, no jugará al tenis".

Este árbol representa un modelo que toma decisiones sobre si alguien jugará al tenis basándose en el clima y la temperatura. Puedes seguir las ramas del árbol para llegar a una predicción.

La construcción de este árbol implica seleccionar las mejores características y establecer umbrales para hacer las preguntas. El objetivo es dividir los datos en grupos homogéneos para hacer predicciones más precisas.

En la práctica, los árboles de decisión pueden volverse más complejos y profundos, y se utilizan para problemas más complicados. Además, los árboles pueden ser parte de algoritmos más avanzados, como bosques aleatorios.

Crear una base de datos simple para trabajar con un árbol de decisión.

Supongamos que quieres predecir si un estudiante aprobará o reprobará un examen basándote en dos características: horas de estudio y asistencia a clases.

Estudiante	Horas de Estudio (X1)	Asistencia a Clases (X2)	Resultado (Y)
Estudiante1	5	10	Aprobado
Estudiante2	3	5	Suspenso
Estudiante3	8	8	Aprobado
Estudiante4	2	3	Suspenso
Estudiante5	7	9	Aprobado

En este ejemplo, "Resultado" es nuestra variable objetivo que queremos predecir. Puedes utilizar estas características para construir un árbol de decisión.

En Kaggle, puedes cargar tu conjunto de datos y utilizar **bibliotecas como scikit-learn para crear y entrenar un modelo de árbol de decisión.** Este modelo puede luego predecir si un estudiante aprobará o reprobará el examen basándose en las horas de estudio y la asistencia a clases.

Seguir los siguientes pasos para trabajar con un árbol de decisión:

1. **Preparar el Entorno:**
   - Abre ***Kaggle*** y ve a la sección "Notebooks".
   - Crea un nuevo cuaderno ("New Notebook") o trabaja en uno existente.

2. **Importar Bibliotecas:**
   - Asegúrate de importar las bibliotecas necesarias. Por ejemplo:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix
```

3. **Cargar y Explorar Datos:**
   - Carga tu conjunto de datos. Puedes utilizar el conjunto de datos que creaste en el ejemplo anterior.

```
# Ejemplo de carga de datos
data = {'Estudiante': ['Estudiante1', 'Estudiante2',
'Estudiante3', 'Estudiante4', 'Estudiante5'],
        'Horas_Estudio': [5, 3, 8, 2, 7],
        'Asistencia_Clases': [10, 5, 8, 3, 9],
        'Resultado': ['Aprobado', 'Suspenso', 'Aprobado',
'Suspenso', 'Aprobado']}
df = pd.DataFrame(data)
```

4. **Preprocesamiento de Datos:**

   - Si es necesario, realiza cualquier preprocesamiento de datos como codificación de variables categóricas, manejo de valores faltantes, etc.

5. **Dividir en Conjunto de Entrenamiento y Prueba:**

   - Divide tus datos en conjuntos de entrenamiento y prueba.

```
X = df[['Horas_Estudio', 'Asistencia_Clases']]
y = df['Resultado']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

6. **Crear y Entrenar el Árbol de Decisión:**

   - Crea un modelo de árbol de decisión y entrénalo.

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

7. **Realizar Predicciones:**

   - Realiza predicciones en el conjunto de prueba.

```
predictions = model.predict(X_test)
```

8. **Evaluar el Modelo:**

- Evalúa el rendimiento del modelo.

```
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy}')
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

Estos son pasos básicos para trabajar con un árbol de decisión en Kaggle.
Puedes adaptarlos según tus necesidades y el conjunto de datos específico que estés utilizando.

Ejemplo de pruebas:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Ejemplo de carga de datos
data = {'Estudiante': ['Estudiante1', 'Estudiante2', 'Estudiante3',
'Estudiante4', 'Estudiante5'],
        'Horas_Estudio': [5, 3, 8, 2, 7],
        'Asistencia_Clases': [10, 5, 8, 3, 9],
        'Resultado': ['Aprobado', 'Suspenso', 'Aprobado', 'Suspenso',
'Aprobado']}
df = pd.DataFrame(data)

X = df[['Horas_Estudio', 'Asistencia_Clases']]
y = df['Resultado']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)


accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy}')
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

   Suspenso       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1

[[1]]
```

Explicación:

Este informe de clasificación y la matriz de confusión indican que el modelo ha alcanzado una precisión del 100%, lo que significa que todas las predicciones en el conjunto de prueba son correctas. Aquí hay una interpretación de cada sección:

➔ **Accuracy: 1.0**

- Todas las predicciones realizadas por el modelo son correctas en comparación con las etiquetas reales.

➔ **Precision, Recall y F1-Score:**

- Dado que tienes solo una clase ("Suspenso"), estas métricas se refieren a esa clase específica.
- Precision: Es la proporción de instancias correctamente clasificadas como "Suspenso" entre todas las instancias predichas como "Suspenso". En este caso, es 1.00, lo que significa que todas las instancias predichas como "Suspenso" son realmente "Suspenso".

- Recall: Es la proporción de instancias correctamente clasificadas como "Suspenso" entre todas las instancias que son realmente "Suspenso". En este caso, también es 1.00, lo que indica que el modelo captura todas las instancias de "Suspenso".

- F1-Score: Es una media armónica de precision y recall. En este caso, es 1.00, lo que indica un equilibrio entre precision y recall.

➔ **Support:**

- Indica cuántas instancias de la clase "Suspenso" hay en el conjunto de prueba. En este caso, hay 1 instancia.

➔ **Matriz de Confusión:**

- La matriz de confusión muestra que hay una predicción correcta para la clase "Suspenso" (1) y no hay falsos positivos ni falsos negativos.

En resumen, el modelo ha funcionado perfectamente en este conjunto de datos, pero ten en cuenta que con conjuntos de datos más grandes y complejos, es crucial evaluar el rendimiento en varios aspectos y considerar otras métricas. La precisión del 100% puede ser indicativa de un posible sobreajuste, especialmente si el conjunto de datos es pequeño.

9. Visualizar el Árbol de Decisión (opcional):

- Puedes visualizar el árbol de decisión para entender cómo toma decisiones.

```
from sklearn.tree import export_text
tree_rules = export_text(model,
feature_names=list(X.columns))
print(tree_rules)
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

   Suspenso       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1
```

```
[[1]]
|--- Horas_Estudio <= 3.50
|   |--- class: Suspenso
|--- Horas_Estudio >  3.50
|   |--- class: Aprobado
```

Resultado del Modelo:

- Clasificó correctamente todas las instancias en el conjunto de prueba.

Precision (Exactitud):

- Todas las instancias que el modelo predijo como "Suspenso" realmente son "Suspenso".

Recall (Recuperación):

- El modelo identificó correctamente todas las instancias de "Suspenso" en el conjunto de prueba.

F1-Score:

- Es una medida general de precisión y recuperación, y en este caso, muestra que el modelo está funcionando muy bien.

Matriz de Confusión:

- Muestra que no hubo errores en las predicciones. No predijo erróneamente "Aprobado" como "Suspenso" ni viceversa.

Resumen:

- El modelo se desempeñó perfectamente para este conjunto de datos pequeño. Sin embargo, al trabajar con conjuntos de datos más grandes, es esencial considerar otras métricas y evitar el sobreajuste. En resumen, el modelo hizo un trabajo excelente en este caso específico, pero es importante ser cauteloso al interpretar resultados y considerar el rendimiento en situaciones más diversas.

Implementación de un árbol de decisión:

Utilizando el conjunto de datos de estudiantes para predecir si un estudiante aprueba o reprueba en función de las horas de estudio y la asistencia a clases.

Explicación del código:

1. **Carga de datos:** Se crea un DataFrame df con información ficticia sobre estudiantes, incluyendo las horas de estudio, la asistencia a clases y el resultado (Aprobado o Suspenso).

```
data = {'Estudiante': ['Estudiante1', 'Estudiante2', 'Estudiante3',
'Estudiante4', 'Estudiante5'],
        'Horas_Estudio': [5, 3, 8, 2, 7],
        'Asistencia_Clases': [10, 5, 8, 3, 9],
        'Resultado': ['Aprobado', 'Suspenso', 'Aprobado', 'Suspenso',
'Aprobado']}
df = pd.DataFrame(data)
```

2. **División de datos:** Los datos se dividen en conjuntos de entrenamiento (X_train, y_train) y prueba (X_test, y_test) utilizando train_test_split.

```
X = df[['Horas_Estudio', 'Asistencia_Clases']]
y = df['Resultado']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

3. **Entrenamiento del modelo:** Se crea y entrena un modelo de árbol de decisión utilizando DecisionTreeClassifier.

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
```

4. **Predicciones y evaluación:** Se realizan predicciones en el conjunto de prueba y se evalúa el rendimiento del modelo utilizando métricas como la precisión, el informe de clasificación y la matriz de confusión.

```
predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy}')
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

5. **Reglas del árbol de decisión:** Se obtienen las reglas del árbol de decisión para visualizar cómo toma decisiones el modelo.

```
from sklearn.tree import export_text
tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)
```

El código anterior es un ejemplo de cómo se puede utilizar un árbol de decisión para clasificar estudiantes según su rendimiento académico. El resultado incluye métricas de evaluación y las reglas del árbol de decisión.

Agregar visualizaciones al código:

Para observar gráficamente cómo el árbol de decisión toma decisiones, puedes utilizar la biblioteca matplotlib y la función plot_tree de sklearn.tree.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt

# Ejemplo de carga de datos
data = {'Estudiante': ['Estudiante1', 'Estudiante2', 'Estudiante3',
'Estudiante4', 'Estudiante5'],
        'Horas_Estudio': [5, 3, 8, 2, 7],
        'Asistencia_Clases': [10, 5, 8, 3, 9],
        'Resultado': ['Aprobado', 'Suspenso', 'Aprobado', 'Suspenso',
'Aprobado']}
df = pd.DataFrame(data)

X = df[['Horas_Estudio', 'Asistencia_Clases']]
y = df['Resultado']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Crear y entrenar el modelo de árbol de decisión
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Realizar predicciones
```

```
predictions = model.predict(X_test)

# Evaluar el rendimiento del modelo
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy}')
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))

# Visualizar el árbol de decisión
plt.figure(figsize=(10, 6))
plot_tree(model, feature_names=X.columns, class_names=model.classes_,
filled=True, rounded=True)
plt.show()
```

Este código adiciona una visualización del árbol de decisión utilizando *plot_tree* después de entrenar el modelo.

Ajusta el tamaño y otros parámetros según tus preferencias.

La visualización del árbol te permitirá entender cómo se toman las decisiones en cada nodo y hoja del árbol.

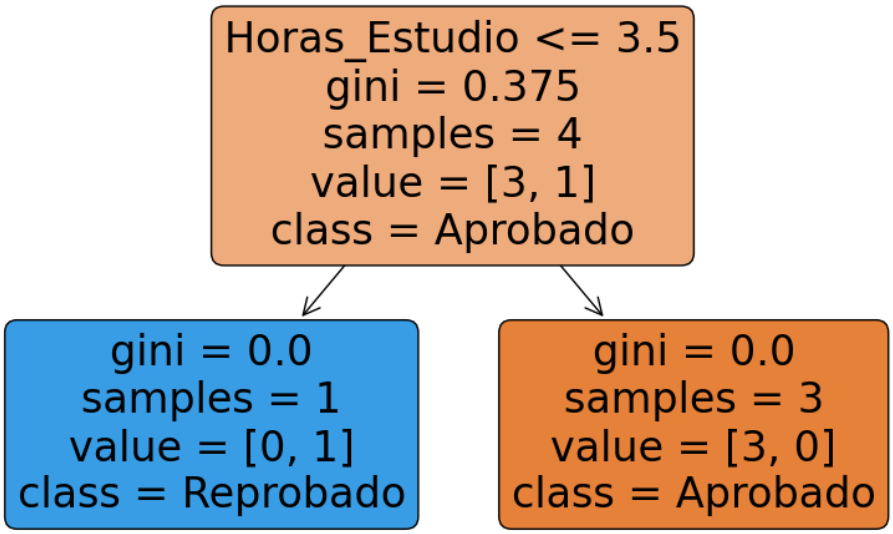
```
Accuracy: 1.0

              precision    recall  f1-score   support

    Suspenso       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1
```

[[1]]



La visualización del árbol de decisión muestra cómo se toman las decisiones en cada nodo del árbol.

Aquí hay una explicación general de lo que significa cada parte:

1. **Node (Nodo):** Cada caja en el gráfico representa un nodo del árbol.
2. **Gini:** Es una medida de la impureza del nodo. Cuanto más bajo sea el valor de Gini, más "puro" es el nodo. En términos simples, un nodo puro significa que todas las muestras en ese nodo pertenecen a la misma clase.
3. **Samples (Muestras):** El número de muestras en ese nodo.
4. **Value (Valor):** Desglose del número de muestras en cada clase en ese nodo.
5. **Feature (Característica):** La característica utilizada para hacer la división en ese nodo.
6. **Threshold (Umbral):** El valor de la característica en el que se realiza la división.
7. **Decision (Decisión):** La decisión que se toma en ese nodo. Por ejemplo, "True" significa que si la muestra cumple con la condición, sigue el subárbol izquierdo; "False" significa que sigue el subárbol derecho.
8. **Class (Clase):** La clase mayoritaria en ese nodo, que se predice para las muestras en ese nodo.

En resumen, el árbol de decisión toma decisiones basadas en características específicas y umbrales para separar las muestras en

clases. Cada nodo representa una pregunta y las ramas son las respuestas a esa pregunta. Al llegar a una hoja (leaf), la clase de esa hoja es la predicción para las muestras que llegan hasta allí.

Para complementar el árbol de decisión, puedes explorar otras tres visualizaciones comunes:

1. **Matriz de Confusión:**
   - Agrega la siguiente sección después de la sección de impresión del árbol de decisión:

```
# Matriz de Confusión
from sklearn.metrics import plot_confusion_matrix
plot_confusion_matrix(model, X_test, y_test,
display_labels=model.classes_, cmap=plt.cm.Blues)
plt.title('Matriz de Confusión')
plt.show()
```

   - Esta visualización te mostrará cuántas predicciones correctas e incorrectas realizó el modelo en cada clase.

2. **Área bajo la Curva ROC (AUC-ROC):**
   - Agrega la siguiente sección después de la sección de impresión del árbol de decisión:

```
# Área bajo la Curva ROC (AUC-ROC)
from sklearn.metrics import plot_roc_curve
plot_roc_curve(model, X_test, y_test)
plt.title('Curva ROC')
plt.show()
```

   - Esta visualización muestra la capacidad del modelo para discriminar entre clases.

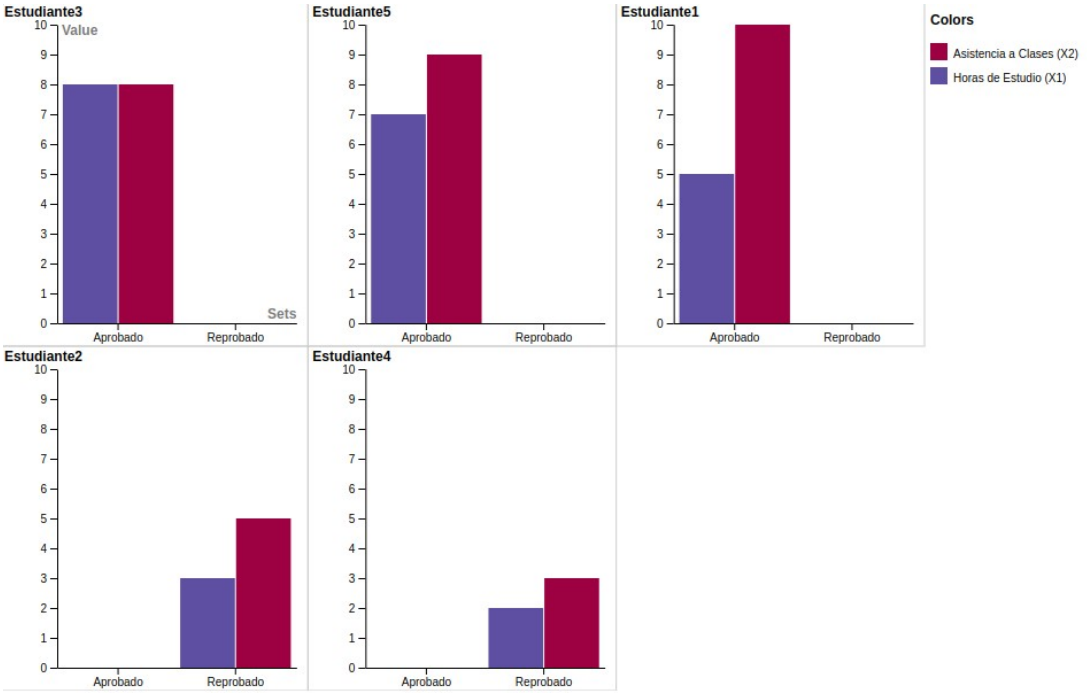
3. **Diagrama de Dependencia Parcial:**
   - Agrega la siguiente sección después de la sección de impresión del árbol de decisión:

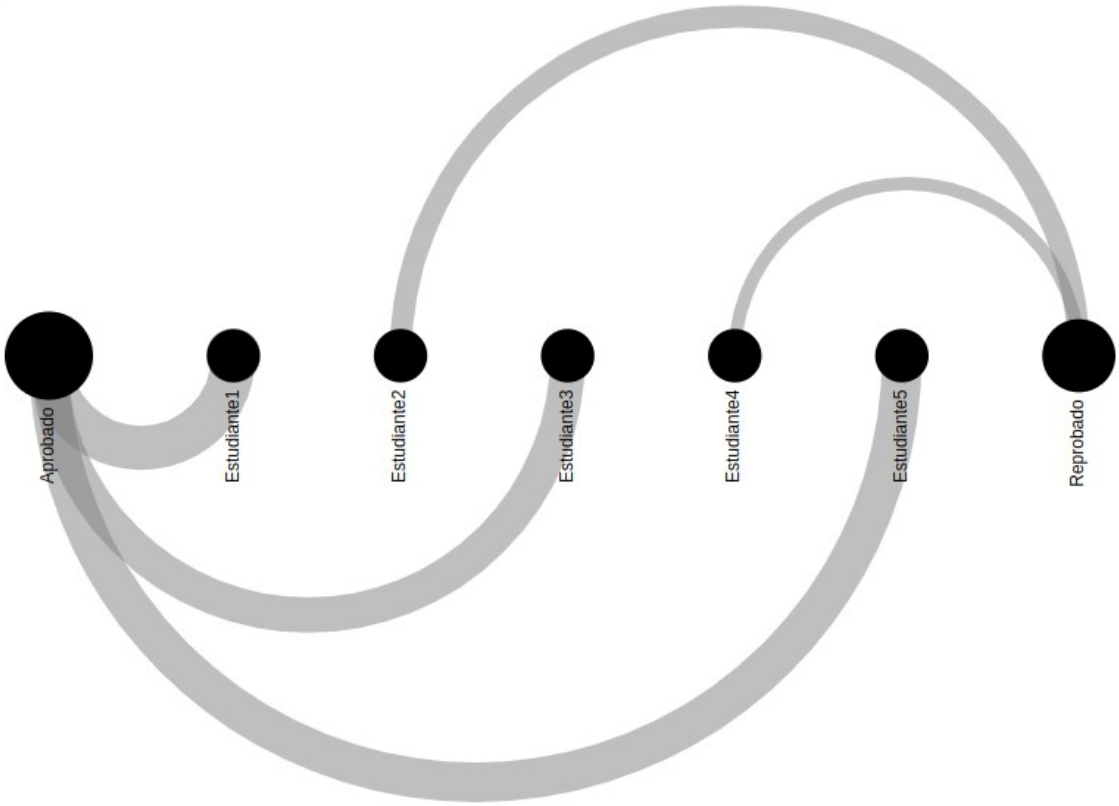
```
# Diagrama de Dependencia Parcial para la primera característica
from sklearn.inspection import plot_partial_dependence
features = [0]  # Índice de la primera característica
plot_partial_dependence(model, X_train, features, grid_resolution=50)
plt.title('Diagrama de Dependencia Parcial')
plt.show()
```

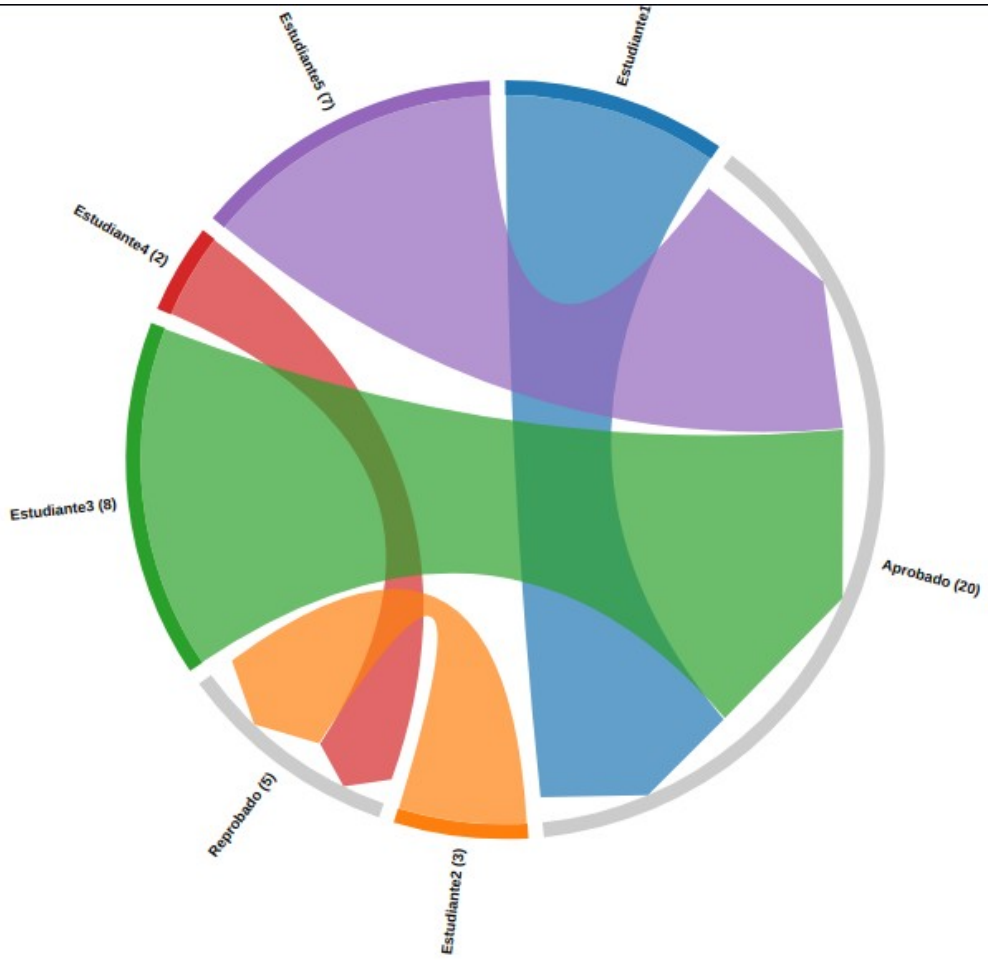
   - Esta visualización ilustra la relación entre una característica específica y la predicción del modelo, manteniendo constantes las demás características.

Asegúrate de agregar estas secciones después de la impresión del árbol de decisión y antes de ejecutar el código completo.
Estas visualizaciones proporcionarán una comprensión más completa del rendimiento y el comportamiento del modelo.

Utilizando **misma base de datos** y otros programas: RAWGraphics




Aprobado
Estudiante1
Estudiante2
Estudiante3
Estudiante4
Estudiante5
Reprobado


Estudiante5 (7)
Estudiante1
Estudiante4 (2)
Estudiante3 (8)
Aprobado (20)
Reprobado (5)
Estudiante2 (3)

con google COLAB:
https://colab.research.google.com/notebooks/charts.ipynb#scrollTo=QSMmdrrVLZ-N

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.inspection import plot_partial_dependence
from sklearn.metrics import plot_roc_curve, plot_confusion_matrix
import matplotlib.pyplot as plt

import pandas as pd

url = "/content/aprobados.csv"
df = pd.read_csv(url)

# Ahora puedes usar el DataFrame 'df' en tu código
print(df.head())


# Ejemplo de carga de datos
data = {'Estudiante': ['Estudiante1', 'Estudiante2', 'Estudiante3',
'Estudiante4', 'Estudiante5'],
'Horas_Estudio': [5, 3, 8, 2, 7],
'Asistencia_Clases': [10, 5, 8, 3, 9],
'Resultado': ['Aprobado', 'Suspenso', 'Aprobado', 'Suspenso', 'Aprobado']}
df = pd.DataFrame(data)

X = df[['Horas_Estudio', 'Asistencia_Clases']]
y = df['Resultado']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Crear y entrenar el modelo de árbol de decisión
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Realizar predicciones
predictions = model.predict(X_test)

# Evaluar el rendimiento del modelo
accuracy = accuracy_score(y_test, predictions)
print(f'Accuracy: {accuracy}')
print(classification_report(y_test, predictions))
```

```
print(confusion_matrix(y_test, predictions))

# Diagrama de Dependencia Parcial para la primera característica
fig, ax = plt.subplots(figsize=(10, 6))
display = plot_partial_dependence(model, X_train, features=[0],
grid_resolution=50, ax=ax)
plt.title('Diagrama de Dependencia Parcial')
plt.show()

# Área bajo la Curva ROC (AUC-ROC)
plot_roc_curve(model, X_test, y_test)
plt.title('Curva ROC')
plt.show()

# Matriz de Confusión
plot_confusion_matrix(model, X_test, y_test, display_labels=model.classes_,
cmap=plt.cm.Blues)
plt.title('Matriz de Confusión')
plt.show()

# Visualizar el árbol de decisión
plt.figure(figsize=(10, 6))
plot_tree(model, feature_names=X.columns, class_names=model.classes_,
filled=True, rounded=True)
plt.show()
```

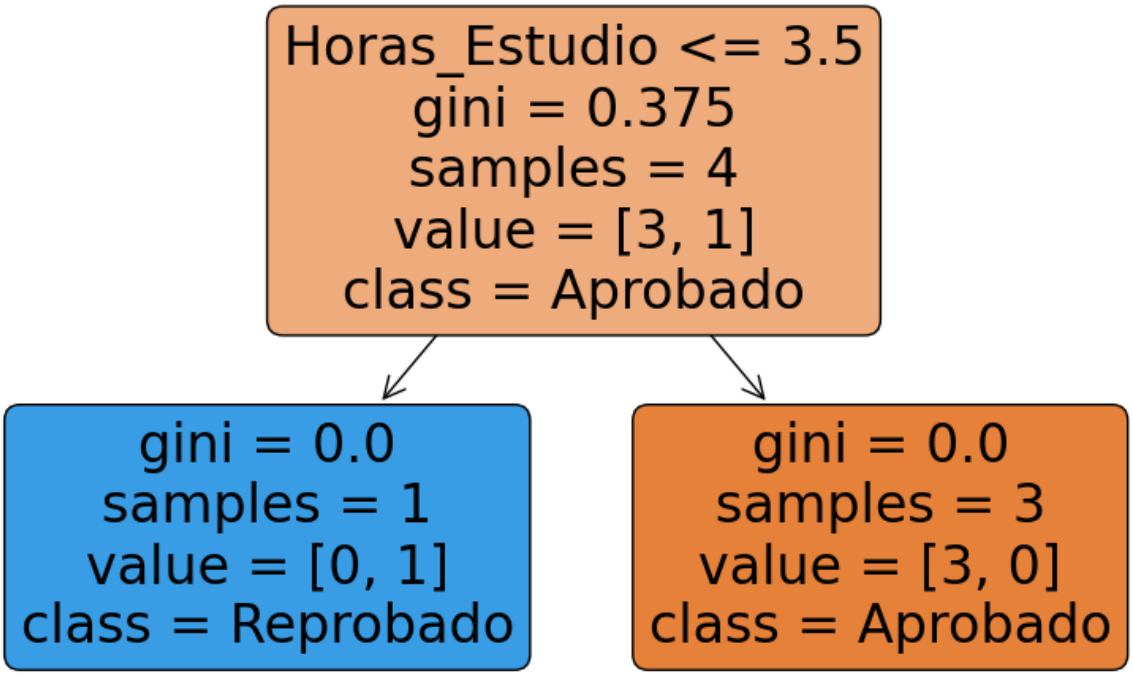
cargando charts:
https://colab.research.google.com/notebooks/charts.ipynb#scrollTo=Eo2bdDQ5GZ76

```
Accuracy: 1.0
              precision    recall  f1-score   support

   Suspenso       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1

[[1]]
```



Conclusión: Probando con diferentes modelos y visualizaciones, al compartir la base de datos la decisión de resultados del árbol es la misma. Seguir con ello.